I need you to help me understand this error and suggest how i can fix it: @prisma/client did not initialize yet. Please run "prisma generate" and try to import it again.
Call Stack
50

Show 46 ignore-listed frame(s)
module evaluation
file:///C:/Users/Nayan%20Curro%20School/Documents/SDP-Lab1/.next/dev/server/chunks/[root-of-the-server]__0xs20qf._.js (48:42)

Show more

This error means that your application is trying to import the Prisma Client, but the client code hasn't been generated yet (or Next.js is trying to use an outdated version of it).

What the error means

When you install Prisma, there are two separate packages:

- `prisma` – the CLI used for development (migrations, generate, etc.)
- `@prisma/client` – the generated client your application actually imports.

When you write something like:

TypeScript

```
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();
```

`@prisma/client` expects generated files to exist inside `node_modules/.prisma`.

If those files don't exist, you'll get:

```
@prisma/client did not initialize yet.
Please run "prisma generate"
```

Common causes

1. You haven't generated the Prisma Client (most common)

Run:

```
Bash
npx prisma generate
```

You should see something like:

```
✓ Generated Prisma Client to ./node_modules/@prisma/client
```

Then restart your Next.js server.

2. Your Prisma schema has errors

If `prisma generate` fails, Prisma won't generate the client.

Run:

```
Bash
npx prisma validate
```

If there are schema errors, fix those first.

3. You installed dependencies but skipped postinstall scripts

Sometimes after cloning a project:

```
Bash
npm install
```

doesn't generate the client.

Simply run:

```
Bash
npx prisma generate
```

4. You deleted `node_modules`

If you recently removed:

```
node_modules
```

or

```
package-lock.json
```

then reinstall:

```
Bash
```

```
npm install
npx prisma generate
```

5. Next.js cache is stale

Sometimes Next caches the old generated client.

Delete:

```
.next
```

Then:

```
Bash
```

```
npm run dev
```

or

```
Bash
```

```
rm -rf .next
npm run dev
```

On Windows:

PowerShell

```
rmdir /s /q .next
npm run dev
```

or simply delete the `.next` folder in File Explorer.

6. Your generated client is missing

Check whether this exists:

```
node_modules
    .prisma
        client
```

If `.prisma/client` doesn't exist, generation didn't happen.

7. Wrong import

Make sure you're importing exactly:

TypeScript

```
import { PrismaClient } from "@prisma/client";
```

Not:

TypeScript

```
import PrismaClient from "@prisma/client"; // ❌
```

or

TypeScript

```
import prisma from "prisma"; // ❌
```

8. Prisma version mismatch

Check:

Bash

```
npm list prisma
npm list @prisma/client
```

They should be the same version.

For example:

```
prisma@6.15.0
@prisma/client@6.15.0
```

If they differ:

Bash

```
npm install prisma@latest @prisma/client@latest
npx prisma generate
```

9. Environment variables

Sometimes Prisma can't generate because your datasource is invalid.

Check that your `.env` contains something like:

```
DATABASE_URL="file:./dev.db"
```

or

```
DATABASE_URL="postgresql://..."
```

Because you're using Next.js

Your stack trace shows:

```
.next/dev/server/app/api/todos/route.js
```

This means the failure occurs while evaluating your API route.

A typical Prisma setup looks like:

TypeScript

```ts
// lib/prisma.ts

import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as {
  prisma?: PrismaClient;
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient();

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

Then:

TypeScript

```ts
import { prisma } from "@/lib/prisma";
```

This pattern avoids creating multiple Prisma clients during development, but it still requires that the client has been generated successfully.

I would try these steps in order

1. Stop the dev server.
2. Run:

Bash

```bash
npx prisma generate
```

3. If that succeeds, delete the `.next` folder.
4. Start the app again:

Bash

```bash
npm run dev
```

5. If `prisma generate` fails, fix the reported schema error.
6. If it succeeds but the error remains, reinstall dependencies:

Bash

```
rm -rf node_modules
npm install
npx prisma generate
npm run dev
```

To narrow it down further

Could you share the following?

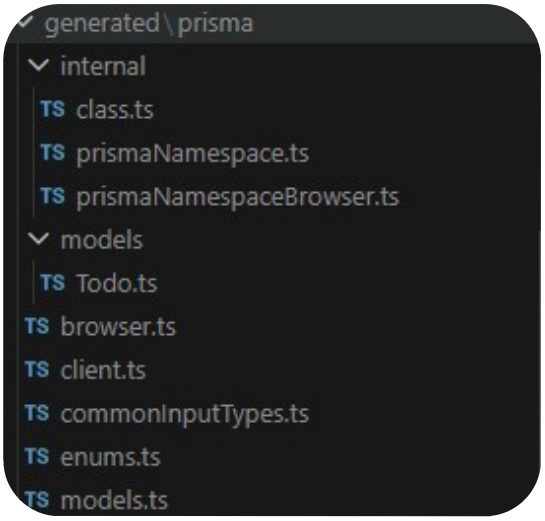
1. The output of:

Bash

```
npx prisma generate
```

2. Your `package.json` (especially the `dependencies` and `devDependencies` sections).
3. Your `prisma/schema.prisma` file.
4. Your Prisma client file (often `lib/prisma.ts` or `src/lib/prisma.ts`).

Those four items will usually reveal exactly why Prisma isn't initializing.


generated \ prisma
internal
class.ts
prismaNamespace.ts
prismaNamespaceBrowser.ts
models
Todo.ts
browser.ts
client.ts
commonInputTypes.ts
enums.ts
models.ts

prisma.config.ts: // This file was generated by Prisma, and assumes you have installed the following:
// npm install --save-dev prisma dotenv
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
  },

Show more

This explains the problem almost completely. The project is in an **inconsistent state**.

What I noticed

Your generated files are:

```
src/generated/prisma/
    client.ts
    browser.ts
    models/
    ...
```

This is **100% the output of the new `prisma-client` generator**.

But your `prisma.ts` is importing:

TypeScript

```
import { PrismaClient } from "@prisma/client";
```

That import belongs to the **old `prisma-client-js` generator**.

That's why TypeScript says:

```
Module '@prisma/client' has no exported member 'PrismaClient'
```

because your project isn't generating that client.

However...

There's another problem.

Your runtime error says:

```
PrismaClient requires a driver adapter...
```

That is **also correct**, because the new generator requires an adapter.

So right now you have **both**:

- old import
- new generator

which don't work together.

I recommend the simplest fix

Unless your lecturer specifically told you to use the new Prisma 7 generator, I would switch back to the classic Prisma client.

Step 1

Open `schema.prisma`.

Replace

prisma

```
generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}
```

with

prisma

```
generator client {
  provider = "prisma-client-js"
}
```

Delete the `output` line entirely.

Step 2

Delete the generated folder:

```
src/generated/prisma
```

Step 3

Run

Bash

```
npx prisma generate
```

Now Prisma should generate into

```
node_modules/@prisma/client
```

instead.

Step 4

Keep your `lib/prisma.ts` exactly as it is:

TypeScript

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = global as {
  prisma?: PrismaClient;
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: ["query", "error", "warn"],
  });

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

Step 5

Run

```bash
npx prisma generate
```

again.

Then

```bash
npm run dev
```

One thing worries me...

You are using

```
Prisma 7.9.1
```

which is **very new**.

I haven't seen many university projects using Prisma 7 yet because the client generation changed significantly.

If your project was created from a tutorial or starter template, it's possible it intentionally uses the new API. In that case, the fix is different (you would keep `provider = "prisma-client"` and add a SQLite driver adapter).

Before we decide, I have one question:

Did you create this project yourself, or was it generated from a tutorial/template?

Also, **how did you create the Prisma project?** For example:

- `npx prisma init`
- `npm create prisma`
- `create-next-app` + Prisma
- A university starter project
- Something else

That will tell me whether we should:

1. **Switch back to the classic Prisma client** (my recommendation for most Next.js apps), or
2. **Keep the new Prisma 7 client** and configure the required SQLite adapter correctly.

I used a template with npm commands. We can switch back to the classic Prisma client.
npm run dev

> todo-app@0.1.0 dev
> next dev

▲ Next.js 16.2.12 (Turbopack)
- Local: http://localhost:3000
- Network: http://100.89.82.35:3000
- Environments: .env

Show more

We're very close. The fact that you're **still getting the driver adapter error** tells me that the project is **still using the new generated client**, even if you've changed the import.

That means either:

- `schema.prisma` wasn't changed successfully, or
- `prisma generate` wasn't run after changing it, or
- Prisma 7 is still generating the new client because of the generator configuration.

Let's verify each step

1. Show me your current `schema.prisma`

I want to confirm it now looks **exactly** like this:

prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Todo {
  id          Int      @id @default(autoincrement())
  title       String
  description String?
  dueDate     DateTime?
  topic       String
  category    String?
  status      String
  createdAt   DateTime @default(now())
  archived    Boolean  @default(false)
  updatedAt   DateTime @updatedAt
}
```

Notice that:

- `provider = "prisma-client-js"` (not `prisma-client`)
- `output` has been removed
- `url = env("DATABASE_URL")` has been added

2. Run these commands

From the root of your project (`Documents/SDP-Lab1`), run:

Bash

```
npx prisma generate
```

Then paste the **entire output**. I want to see whether it says something like:

```
✓ Generated Prisma Client to ./node_modules/@prisma/client
```

or if it still generates into `src/generated/prisma`.

3. Check whether the old generated folder still exists

Does this folder still exist?

```
src/generated/prisma
```

If yes, delete it completely.

4. Check the installed Prisma version

Run:

Bash

```
npx prisma --version
```

and paste the output.

One thing I suspect

You're on **Prisma 7.9.1**, which is very new. If `prisma-client-js` isn't behaving as expected, there may be changes in Prisma 7 that are making the migration path different from earlier versions.

If that's the case, I may recommend **downgrading to Prisma 6**, which is stable, widely documented, and works perfectly with the classic `PrismaClient` API.

For example:

Bash

```
npm install prisma@6 @prisma/client@6
```

That would restore the familiar behavior used in most tutorials and examples.

Before we downgrade, though, I'd like to see:

1. Your **current** `schema.prisma` .
2. The output of:

Bash

```
npx prisma generate
```

3. The output of:

```bash
npx prisma --version
```

Those three pieces will tell us whether this is just a configuration issue or whether Prisma 7 is the root cause.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url = env("DATABASE_URL")
}

model Todo {
```

Show more

Excellent—that output identifies the root cause.

The root cause

You're using **Prisma 7.9.1**.

Prisma 7 fundamentally changed how the client works:

- ❌ `url` is no longer allowed in `schema.prisma`.
- ❌ The classic `new PrismaClient()` no longer works without an adapter.
- ✅ Connection configuration moved to `prisma.config.ts`.
- ✅ Driver adapters are now the default architecture.

This means we **cannot simply switch back to the old API while staying on Prisma 7**.

I recommend the simpler option

Since this is a SQLite Todo app and you want the familiar Prisma API, **downgrade to Prisma 6**.

This is the version used by the majority of tutorials and examples, and it matches the code you've already written.

Step 1: Remove the current Prisma packages

Bash

```
npm uninstall prisma @prisma/client
```

Step 2: Install Prisma 6

Bash

```
npm install prisma@6 @prisma/client@6
```

Step 3: Delete the Prisma 7 config

Delete:

```
prisma.config.ts
```

Prisma 6 doesn't use it.

Step 4: Restore `schema.prisma`

Use:

prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Todo {
  id          Int      @id @default(autoincrement())
  title       String
  description String?
  dueDate     DateTime?
  topic       String
  category    String?
  status      String
  createdAt   DateTime @default(now())
  archived    Boolean  @default(false)
  updatedAt   DateTime @updatedAt
}
```

Step 5: Generate the client

Bash

```
npx prisma generate
```

You should now see something like:

```
✓ Generated Prisma Client to ./node_modules/@prisma/client
```

Step 6: If you don't already have a database

Run:

Bash

```
npx prisma migrate dev --name init
```

or, if you're only syncing the schema:

Bash

```
npx prisma db push
```

Step 7: Keep your `lib/prisma.ts`

Your existing file is exactly what Prisma 6 expects:

TypeScript

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = global as {
  prisma?: PrismaClient;
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: ["query", "error", "warn"],
  });

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

There's one unrelated warning

Next.js also printed:

```
Warning: Next.js inferred your workspace root...
Detected additional lockfiles...
```

You have **two** `package-lock.json` **files**:

```
C:\Users\Nayan Curro School\package-lock.json
```

and

```
C:\Users\Nayan Curro School\Documents\SDP-Lab1\package-lock.json
```

That won't cause the Prisma error, but it can confuse Next.js. If you're only working in `Documents\SDP-Lab1`, you can usually remove the parent `package-lock.json` (after confirming it's not part of another project), or configure the Turbopack root explicitly.

I think downgrading is the cleanest path here. You're trying to use the classic Prisma API, while Prisma 7 is designed around the new adapter-based architecture. Rather than rewriting your application to use adapters, Prisma 6 lets you use the API your code already expects with minimal changes. After installing Prisma 6 and regenerating the client, the `PrismaClient` import and constructor should work as intended.